Manual da tela de regulação — OmniLeito

- Data: 19/10/2026
- Página: `http://localhost:8085/`
- Complementa `docs/roteiro-de-testes.md`. O curl daquele roteiro continua válido: a tela chama as mesmas APIs.

A tela é a interface do regulador. O navegador fala só com o API Gateway. Cada cartão mostra a API, quem gera a resposta ou a mensagem e quem consome.

1. O que precisa estar no ar

Os quatro containers do `docker compose` e os seis processos:

Processo	Porta
API Gateway	8085
MS-Identidade	8092
MS-Ocorrencias	8093
MS-Frota	8094
MS-Leitos	8095
MS-Notificacoes	8096

A carga inicial tem duas ambulâncias `DISPONIVEL` (`ABC-1234` e `XYZ-5678`) e, em cada especialidade, um leito `LIVRE` e um `OCUPADO`. Se uma passagem anterior gastou UTI ou ambulância, use o cartão **Estado inicial**. O `DROP DATABASE` da seção 2 do roteiro continua valendo quando o schema precisa nascer de novo.

Abra `http://localhost:8085/` e atualize a página se o navegador mostrar a versão antiga.

2. Como a página funciona

No alto, **OmniLeito / Painel de Controle**. À direita, o estado da sessão.

Sem login, o texto é `Sem token. Entre para chamar a regulação.` Depois do `200`, passa a `Token guardado nesta aba.` O token fica em `sessionStorage` e vale só nessa aba. Fechar a aba pede um novo login. As chamadas seguintes enviam `Authorization: Bearer`.

Cada cartão tem:

- o código do requisito, no topo
- o título da ação
- o resumo **API / Gera / Consome**
- os campos e botões
- a área de resposta, em HTTP e JSON

`HTTP 0` com `SEM_CONEXAO` significa que o gateway na porta 8085 não respondeu. `401` e `TOKEN_INVALIDO` significam que o token falta ou venceu: entre de novo.

O campo **Ocorrência**, no cartão Despachar, recebe o UUID da última abertura. Segundo despacho, notificações e o prazo usam esse mesmo valor. Troque o texto do campo se quiser consultar outra ocorrência.

3. Ordem de uma passagem

1. Entrar.
2. Consultar a ocupação, para ver a carga antes de gastar recurso.
3. Escolher o modo do hospital.
4. Abrir a ocorrência.
5. Despachar e ler o acompanhamento.
6. Consultar notificações.
7. Seguir o cenário: segundo despacho, prazo, outro leito, falta de leito, circuito ou o teste das filas da seção 5.

O modo do hospital vale para a próxima reserva e permanece até outro clique ou até o MS-Leitos subir de novo. Na subida, o argumento `--omnileito.reserva.resultado` é o valor inicial. Sem argumento, o modo começa aleatório.

4. Cartões

Entrar — RF01

Usuário e senha da carga: `regulador` / `regulador`.

Peça	Papel
API	`POST /api/auth/login`
Gera	MS-Identidade devolve o JWT. Não há fila
Consome	API Gateway encaminha

Esperado: HTTP `200`, `token` e `tipo` `Bearer`. Usuário ou senha errados: HTTP `401`, `CREDENCIAL_INVALIDA`.

Abrir ocorrência — RF02 a RF04

Informe paciente, local, prioridade (`VERMELHA`, `AMARELA` ou `VERDE`) e tipo de leito (`UTI`, `EMERGENCIA` ou `ENFERMARIA`). O despacho usa o tipo gravado aqui e não pede outro.

Peça	Papel
API	`POST /api/ocorrencias`
Gera	MS-Ocorrencias grava `ABERTA`. A abertura não publica mensagem
Consome	API Gateway encaminha

Esperado: HTTP `201`, `status` `ABERTA`, `ambulanciaId` e `leitoId` nulos, e um UUID em `id`. Esse UUID entra sozinho no campo Ocorrência.

A abertura continua respondendo `201` com o MS-Leitos parado. Ela não passa pelo circuito.

Ocupação — RF17 e RF18

Ambulâncias disponíveis lista só as `DISPONIVEL`. **Leitos livres** usa o tipo escolhido ao lado e lista só os `LIVRE` dessa especialidade. Leito `OCUPADO` e leito de outro tipo não aparecem.

Peça	Papel
API	`GET /api/ambulancias/disponiveis` e `GET /api/leitos/disponiveis`
Gera	MS-Frota e MS-Leitos. Cada um lê o próprio Redis e, sem chave, o MySQL
Consome	API Gateway. A consulta de leitos passa pelo circuito

Na carga limpa, ambulâncias devolve `ABC-1234` e `XYZ-5678`. UTI devolve `L-UTI-01`. Lista `[]` com HTTP `200` é especialidade sem vaga livre, não é falha do circuito. O `503` só aparece com o MS-Leitos parado.

A chave do Redis nasce nessa primeira leitura. O nome dela (`frota:disponiveis` ou `leitos:disponiveis:UTI`) se confere no Redis, como na seção 6 do roteiro. A tela mostra a lista, não a chave.

Modo do hospital — RF13

Quatro botões: **Aceite**, **Recusa**, **Silêncio** e **Aleatório**.

Peça	Papel
API	`GET` e `PUT /api/leitos/demonstracao/resultado`
Gera	MS-Leitos guarda o modo e usa na próxima reserva
Consome	API Gateway encaminha. Sem fila

O texto abaixo dos botões passa a `Modo atual: ACEITE`, `RECUSA`, `SILENCIO` ou `aleatório`. Trocar o modo não reinicia a porta 8095. Com o MS-Leitos parado, o `PUT` também cai no circuito.

O modo fica valendo para toda reserva seguinte, inclusive a busca automática depois de recusa ou prazo. Volte a **Aleatório**, ou ao modo do próximo cenário, quando terminar.

Despachar — RF06 a RF12

Usa o UUID do campo Ocorrência. **Despachar** envia o pedido. **Consultar** lê o estado uma vez, sem iniciar saga.

Peça	Papel
API	`POST /api/ocorrencias/{id}/despachos` e `GET /api/ocorrencias/{id}`
Gera	MS-Ocorrencias publica `saga.alocar-ambulancia`. MS-Frota responde em `saga.ambulancia-alocada`. MS-Ocorrencias publica `saga.reservar-leito`. MS-Leitos responde em `saga.leito-reservado`
Consome	MS-Frota, MS-Leitos e MS-Ocorrencias, nessa ordem. No sucesso, MS-Ocorrencias publica `saga.atualizar-ambulancia` e `notificacao.registrar`, consumidas por MS-Frota e MS-Notificacoes

O primeiro bloco é o HTTP do `POST`. `202` só diz que a saga começou. Em seguida a tela consulta a ocorrência a cada 2 segundos, por até 15 segundos, no bloco **Acompanhamento**.

No sucesso, com leito livre e modo aceite:

Campo	Valor
status	`DESPACHADA`
ambulanciaId	`1` na primeira ambulância da carga (`ABC-1234`)
leitoId	id de `L-UTI-01`, `L-EMG-01` ou `L-ENF-01`, conforme o tipo
unidadeNome	`Hospital Central` ou `UPA Norte`

`EM_DESPACHO` no meio do acompanhamento é a saga ainda andando. `FALHA_DESPACHO` com ambulância e leito nulos é o fim da primeira busca sem vaga ou sem ambulância.

Campo vazio: a área de resposta pede para abrir uma ocorrência. UUID que não existe: HTTP `404`, `OCORRENCIA_NAO_ENCONTRADA`.

Segundo despacho — RF07

Repete o `POST /despachos` do UUID que está no campo.

Peça	Papel
API	`POST /api/ocorrencias/{id}/despachos`
Gera	MS-Ocorrencias responde `200` com a mesma ambulância e o mesmo leito
Consome	API Gateway. Esta repetição não publica fila nova

Faça isso depois de um despacho que já chegou a `DESPACHADA` ou a `FALHA_DESPACHO`. No sucesso, a ambulância que ainda estava `DISPONIVEL` continua `DISPONIVEL`. Uma ocorrência `FALHA_DESPACHO` repetida responde `200` e permanece `FALHA_DESPACHO`: a saga não recomeça.

Ocorrência `CANCELADA` ou `CONCLUIDA` responde `409`, `OCORRENCIA_NAO_ABERTA`. Este fluxo não conduz a esses dois status.

Notificações — RF24

Peça	Papel
API	`GET /api/notificacoes?ocorrenciaId=`
Gera	MS-Ocorrencias ou MS-Leitos publicam `notificacao.registrar`
Consome	MS-Notificacoes grava no Mongo e devolve a lista. API Gateway encaminha a consulta

Os documentos ficam no MongoDB `omnileito_notificacoes`, coleção `notificacoes`, não no MySQL. O horário `ocorridaEm` vem em UTC, com sufixo `Z`.

Tipo	Quando aparece	Texto
`DESPACHO`	Leito reservado e ocorrência `DESPACHADA`	Ambulância, código do leito e unidade
`EXPIRACAO`	Reserva sem confirmação venceu o prazo	`Reserva do leito ... expirou.`
`FALTA_LEITO`	Busca seguinte sem nenhum leito livre	`Não há leito livre da especialidade ...`

Uma ocorrência que terminou em `FALHA_DESPACHO` na primeira busca não ganha `DESPACHO`.

Prazo do aceite — critério 6

Use depois de um despacho em modo **Aceite** que ficou `DESPACHADA`.

Peça	Papel
API	`GET /api/notificacoes?ocorrenciaId=` depois dos 2 minutos
Gera	O job do MS-Leitos lê o MySQL a cada 15 segundos. Reserva confirmada não publica `saga.leito-expirado`
Consome	MS-Notificacoes só recebe `EXPIRACAO` quando o prazo vence uma reserva sem confirmação

Iniciar marcador de 2 minutos conta na tela. Ao chegar a zero, consulta as notificações do UUID do campo. O esperado no aceite é `Nenhuma EXPIRACAO nesta ocorrência.` A prova de que o leito ficou `confirmado` verdadeiro continua no MySQL, como na seção 7 do roteiro: a lista de leitos livres não traz esse campo.

No silêncio, o mesmo marcador deve encontrar `EXPIRACAO`, e a saga reserva o mesmo leito de novo se ele for o único livre.

Outro leito — RF22

A carga tem um único livre por especialidade, então a recusa repetiria o mesmo código. Este cartão cria um livre a mais.

Peça	Papel
API	`POST /api/leitos/demonstracao/leito-extra` e `DELETE /api/leitos/demonstracao/leito-extra/L-UTI-03`
Gera	MS-Leitos cria ou remove o leito e apaga o cache da especialidade
Consome	Na recusa, MS-Ocorrencias publica `saga.reservar-leito` e MS-Leitos prefere o outro leito livre

Criar L-UTI-03 espera HTTP `201`, status `LIVRE`, unidade `Hospital Central`. A ocupação de UTI passa a listar esse código junto com o outro livre. **Apagar L-UTI-03** espera HTTP `204` e só funciona com o leito `LIVRE`.

`409 LEITO_DA_CARGA` protege `L-UTI-01`, `L-UTI-02` e os códigos de emergência e enfermaria. `409 LEITO_NAO_LIVRE` aparece se `L-UTI-03` estiver reservado: espere a reserva acabar, ou encerre o ciclo, e apague quando voltar a livre. `409 LEITO_JA_EXISTE` aparece no segundo criar sem apagar o primeiro.

Falta de leito — RF23

Sem nenhum livre na primeira tentativa, o desfecho é `FALHA_DESPACHO` e a ambulância volta a `DISPONIVEL`. `FALTA_LEITO` é outro caminho: a ambulância já está na ocorrência e a busca seguinte não acha vaga. A carga sempre devolve o leito recusado a `LIVRE`, e a saga o reserva de novo. A trava existe para a banca atravessar esse ramo uma vez.

Peça	Papel
API	`POST /api/leitos/demonstracao/proxima-busca-sem-vaga`
Gera	MS-Leitos, na tentativa seguinte, publica `saga.leito-reservado` com `SEM_LEITO_COMPATIVEL`
Consome	MS-Ocorrencias publica `notificacao.registrar` do tipo `FALTA_LEITO`. MS-Notificacoes grava

A trava nasce desligada a cada subida do MS-Leitos. Armar espera HTTP `204`. Ela vale uma busca de UTI e só na tentativa maior que 1. A tentativa 1 reserva normalmente.

Não há botão para desarmar. Se armou e não despachou, a próxima recusa ou expiração de UTI consome a trava.

Circuito — RF05

Peça	Papel
API	`GET /api/leitos/disponiveis` e `POST /api/ocorrencias`
Gera	Com o MS-Leitos parado, o API Gateway responde `503`. A abertura continua no MS-Ocorrencias
Consome	O circuito vale só para `/api/leitos`. Login, abertura e despacho não entram nele

Pare só o processo da porta 8095 com Ctrl+C. Ctrl+Z suspende e a porta continua ocupada.

1. **Consultar UTI** duas vezes. Esperado: HTTP `503`, `codigo` `SERVICO_LEITOS_INDISPONIVEL`.
2. **Abrir ocorrência agora**, no mesmo momento. Esperado: HTTP `201`. O paciente gravado é `Teste Circuito`, local `Rua do Circuito, 1`, prioridade `VERDE`, tipo `ENFERMARIA`. O UUID também entra no campo Ocorrência.
3. Suba o MS-Leitos de novo. Depois da janela de 10 segundos do circuito aberto, **Consultar UTI** volta a `200`.

Estado inicial

Devolve a carga sem parar os processos e sem apagar o usuário `regulador`. Use com a saga já parada. O `DROP DATABASE` da seção 2 do roteiro continua sendo o retorno quando o schema precisa nascer de novo.

Peça	Papel
API	`POST /api/ocorrencias/demonstracao/reinicio`, `POST /api/ambulancias/demonstracao/reinicio`, `POST /api/leitos/demonstracao/reinicio` e `POST /api/notificacoes/demonstracao/reinicio`
Gera	MS-Ocorrencias apaga ocorrências e esvazia as oito filas. MS-Frota devolve `ABC-1234` e `XYZ-5678` a `DISPONIVEL` e apaga o cache. MS-Leitos restaura os seis leitos, remove extra como `L-UTI-03`, solta a trava e volta o modo a aleatório. MS-Notificacoes esvazia `notificacoes` e `auditoria`
Consome	API Gateway encaminha as quatro chamadas. A de leitos passa pelo circuito

Reiniciar no estado inicial espera `HTTP 204` em cada chamada. A tela repete a de ocorrências no fim, para esvaziar fila que tenha chegado no meio. Em seguida limpa o UUID e as respostas dos outros cartões. O token da aba permanece.

Se o MS-Leitos estiver parado, a chamada de leitos responde `503` e a carga de leitos não volta. As outras seguem.

O Estado inicial também religa os listeners depois de esvaziar as filas. Um acúmulo interrompido não deixa a regulação surda.

Filas

Dois ensaios no mesmo cartão. O painel do RabbitMQ fica em `http://localhost:15672`, `guest` / `guest`, aba Queues. Atualize essa aba durante o ensaio.

Peça	Papel
API	`POST /api/ocorrencias/demonstracao/rajada`, `POST /api/ocorrencias/demonstracao/filas/acumulo`, `POST /api/ocorrencias/demonstracao/filas/esvaziar` e as pausas e retomadas de ocorrências, frota, leitos e notificações
Gera	A rajada abre e despacha pela saga. O acúmulo publica a quantidade pedida em cada uma das oito filas
Consome	Na rajada, os listeners de sempre. No acúmulo, ninguém consome até **Esvaziar e religar**

Rajada, padrão 20 e teto 100. Abre ocorrências `Carga Fila` em enfermaria e despacha, com 200 ms entre uma e outra. As taxas incoming e deliver/get sobem. No fim as filas voltam a zero e a carga fica gasta. Use **Estado inicial** em seguida.

Pausar e acumular, padrão 40 e teto 200. Pausa os quatro consumidores sem derrubar a porta. Se as quatro pausas forem `204`, publica. O refresh do RabbitMQ mostra Ready igual à quantidade nas oito filas, e esse número permanece. A varredura de leitos, que publicaria uma retentativa a cada 15 segundos, também fica parada nessa pausa. Se uma pausa falhar, o acúmulo não começa e quem já tinha pausado é religado.

Esvaziar e religar apaga as oito filas e só então religa os consumidores. As mensagens de `inspecao-fila` não entram na saga.

Quantidade fora da faixa: HTTP `400`, `CAMPO_INVALIDO`.

O passo a passo, a leitura das colunas do RabbitMQ e o que fazer quando o número foge do esperado estão na seção 5. O plano de onde saiu esse ensaio é a seção 9 de `docs/plano-carga-das-filas.md`.

5. Teste das filas no RabbitMQ

O cartão **Filas** fica depois de Estado inicial. O login é `regulador` / `regulador`. O RabbitMQ Management fica em outra aba: `http://localhost:15672`, `guest` / `guest`, **Queues and Streams**. O painel não abre essa aba. Quem testa atualiza a tabela no momento certo.

A coluna State `running` significa que a fila existe. Ela permanece `running` com o listener pausado. O sinal de pausa é Ready parado e deliver/get em `0.00/s`.

Há dois ensaios. A rajada prova a saga. O acúmulo prova a foto de fila cheia. Um não substitui o outro. Faça o acúmulo primeiro: ele não gasta ambulância nem leito.

O que a rajada faz

Rajada pede uma quantidade, padrão 20, teto 100. Para cada unidade, o MS-Ocorrencias abre uma ocorrência e chama o despacho que a regulação já usa.

Campo da ocorrência	Valor
Paciente	`Carga Fila 1`, `Carga Fila 2`, e assim por diante
Local	`Rua da Fila`
Prioridade	`VERDE`
Tipo de leito	`ENFERMARIA`

Entre um despacho e o seguinte há 200 ms. A resposta do cartão fecha só no fim: HTTP `200` e `{"quantidade": 20}`. Esse `200` diz que as aberturas e os despachos foram pedidos. O estado final de cada ocorrência aparece na ocupação e no MySQL, não nesse JSON.

Os listeners ficam ligados. Cada mensagem entra e sai. Ready, Unacked e Total ficam em `0` no fim. A prova é a coluna **Message rates**:

Coluna	O que significa na rajada
incoming	Alguém publicou naquela fila na janela recente
deliver / get	Um consumidor recebeu mensagem na janela recente
ack	O consumidor confirmou a mensagem
Ready	Mensagem ainda esperando. No fim da rajada, o valor esperado é `0`

A taxa do management é uma média da janela recente. Um pico de vinte mensagens em poucos segundos continua aparecendo, por exemplo como `0.40/s`, depois que Ready já voltou a zero. Mais um refresh, com o sistema quieto, devolve `0.00/s`.

A semente tem duas ambulâncias e um leito livre de enfermaria. As primeiras ocorrências percorrem alocação, reserva e notificação. As seguintes encontram a frota esgotada, passam por `saga.alocar-ambulancia` e `saga.ambulancia-alocada` e terminam em `FALHA_DESPACHO`. Esse fim é o teste da frota esgotada.

`saga.leito-expirado` só recebe mensagem quando uma reserva sem confirmação completa dois minutos. Uma rajada que acabou de terminar deixa essa taxa em `0.00/s`. `notificacao.registrar` sobe nas ocorrências que publicaram despacho; se o refresh vier depois da janela, a taxa já voltou a `0.00/s` e o documento está no Mongo.

`saga.retentar-busca-leito` continua se movendo com o MS-Leitos no ar. A cada 15 segundos a varredura publica uma retentativa para cada especialidade que ainda tem leito livre, e o MS-Ocorrencias consome. Ready segue em `0`. Essa fila é a única que pode mostrar deliver/get e ack acima de zero com o painel já em HTTP `200`.

O que o acúmulo faz

Pausar e acumular pede uma quantidade, padrão 40, teto 200. A ordem é fixa:

1. Pausa o listener do MS-Ocorrencias (`saga.ambulancia-alocada`, `saga.leito-reservado`, `saga.leito-expirado`, `saga.retentar-busca-leito`).
2. Pausa o listener do MS-Frota (`saga.alocar-ambulancia`, `saga.atualizar-ambulancia`).

3. Pausa o listener do MS-Leitos (`saga.reservar-leito`) e a varredura de 15 segundos.
4. Pausa o listener do MS-Notificacoes (`notificacao.registrar`).
5. Só com as quatro respostas em `204`, publica a quantidade em cada uma das oito filas.

O processo Java de cada porta continua no ar. A pausa da porta 8095 não abre o circuito. Se uma pausa não for `204`, o acúmulo não publica e o cartão religa quem já tinha pausado. A área de resposta termina com `Acúmulo não começou.`

Cada mensagem usa o formato que o listener daquela fila já conhece. O `ocorrenciaId` é `inspecao-fila`, que não é ocorrência aberta. A publicação existe para o broker contar. Ela não deve ser entregue.

Com os listeners parados, o refresh mostra nas oito linhas Ready igual à quantidade, Unacked `0` e Total igual à quantidade. incoming sobe na publicação e depois volta a `0.00/s`, com Ready parado. A varredura também fica parada: sem isso, `saga.retentar-busca-leito` ganharia mensagem nova a cada 15 segundos e a foto deixaria de bater com a quantidade.

Esvaziar e religar apaga as oito filas e só então religa os quatro listeners e a varredura. As mensagens de `inspecao-fila` não entram na saga. Ambulância e leito permanecem como estavam.

Preparar

1. Confira as seis portas. Entre no painel com `regulador` / `regulador`.
2. Abra `http://localhost:15672` na aba Queues.
3. No painel, **Reiniciar no estado inicial**. Esperado: cinco linhas `HTTP 204` e o texto de carga restaurada.
4. Atualize o RabbitMQ. As oito filas em Ready `0`. `saga.retentar-busca-leito` pode mostrar uma taxa baixa: é a varredura, com Ready ainda em `0`.

Passo a passo do acúmulo

1. Deixe a quantidade do acúmulo em `40`. Clique **Pausar e acumular**.
2. Esperado no cartão, nesta ordem: `Ocorrências pausa: HTTP 204`, `Frota pausa: HTTP 204`, `Leitos pausa: HTTP 204`, `Notificações pausa: HTTP 204`, `Acúmulo: HTTP 204`.
3. Atualize o RabbitMQ. As oito filas com Ready `40`, Unacked `0`, Total `40`. State continua `running`.
4. Espere mais de 15 segundos e atualize de novo. Ready continua `40` nas oito, inclusive em `saga.retentar-busca-leito`. incoming volta a `0.00/s`.
5. No painel, ocupação: `ABC-1234` e `XYZ-5678` seguem `DISPONIVEL`. O acúmulo não alocou ambulância.
6. Clique **Esvaziar e religar**. Esperado: `Esvaziar: HTTP 204`, as quatro retomadas em `204` e o texto de consumidores religados.
7. Atualize o RabbitMQ. As oito filas em zero.
8. Abra uma ocorrência qualquer e despache. A saga percorre de novo. A pausa não ficou presa. Em seguida, **Estado inicial**, para a rajada achar a semente limpa.

Passo a passo da rajada

1. Carga limpa, pelo Estado inicial do passo anterior.
2. Deixe a quantidade da rajada em `20`. Deixe a aba do RabbitMQ visível. Clique **Rajada**.

3. Enquanto o botão estiver desabilitado e o cartão disser que a rajada está em andamento, atualize Queues duas ou três vezes.
4. Durante esses refreshes, incoming, e em seguida deliver/get e ack, passam de `0.00/s` em `saga.alocar-ambulancia` e `saga.ambulancia-alocada`. `saga.reservar-leito`, `saga.leito-reservado`, `saga.atualizar-ambulancia` e `notificacao.registrar` sobem enquanto ainda houver ambulância e leito. Ready pode piscar acima de zero. O número parado não é a prova deste ensaio.
5. O cartão fecha com HTTP `200` e `{"quantidade": 20}`.
6. Atualize o RabbitMQ de novo. Ready, Unacked e Total em `0`. A média ainda pode mostrar `0.40/s` em incoming nas filas que a saga tocou. `saga.retentar-busca-leito` pode manter deliver/get e ack acima de zero. `saga.leito-expirado` fica em `0.00/s`.
7. Ocupação: as duas ambulâncias saíram de `DISPONIVEL`. O leito livre de enfermaria foi usado. Parte das ocorrências `Carga Fila` fica `DESPACHADA` e o restante `FALHA_DESPACHO`.
8. **Reiniciar no estado inicial**. Cinco `HTTP 204`. Ocupação de novo com as duas ambulâncias e um livre por especialidade. Mais um refresh no RabbitMQ: Ready em `0` e as taxas a caminho de `0.00/s`.

Uma rajada já terminada, com HTTP `200` e Ready em zero, ainda pode mostrar `0.40/s`. Essa taxa é a média do pico que já passou. Para ver a taxa subir ao vivo, o refresh tem que acontecer com o botão **Rajada** ainda desabilitado. A foto de Ready `40` é **Pausar e acumular**.

Quando o resultado foge do esperado

O que aparece	O que fazer
Alguma pausa em HTTP `504` e o texto `Acúmulo não começou.`	O acúmulo não publicou. Quem chegou a pausar foi religado. Repita **Pausar e acumular** com os seis processos no ar
Acúmulo em `204`, mas Ready volta a zero na hora	O listener daquela fila ainda consumiu. Confira se a pausa daquele serviço foi `204` antes da publicação
Ready sobe sozinho em `saga.retentar-busca-leito` durante a pausa	A varredura de leitos ainda está publicando. A pausa do MS-Leitos precisa ser a que também segura essa varredura. **Esvaziar e religar** e repetir o acúmulo
Rajada em HTTP `400`	A quantidade está fora de 1 a 100
Acúmulo em HTTP `400`	A quantidade está fora de 1 a 200
Leitos pausa em HTTP `503`	O processo da porta 8095 está parado. Suba o MS-Leitos. O circuito de consulta continua no limite de 2 segundos; a rajada e a pausa usam o tempo maior do gateway
Depois da rajada a ocupação mostra a frota gasta	Esperado. **Estado inicial** restaura

6. Cenários pela tela

Comece cada bloco com carga limpa, exceto quando o passo disser para continuar na mesma carga. Entre antes do primeiro cartão.

Aceite, segundo despacho e prazo

1. Modo **Aceite**.
2. Ocupação: duas ambulâncias e `L-UTI-01` livre.
3. Abra uma UTI. Despache. Acompanhe até `DESPACHADA`, ambulância 1, `L-UTI-01`, `Hospital Central`.
4. Ocupação de UTI: `[]`. Ambulâncias: só `XYZ-5678`.
5. Notificações: um `DESPACHO`.
6. **Segundo despacho**: HTTP `200`, os mesmos ids. `XYZ-5678` segue disponível.
7. Marcador de 2 minutos. Nenhuma `EXPIRACAO` nova.

Primeira busca sem leito

Continue após o aceite, com `L-UTI-01` já reservado. Abra outra UTI, com outro paciente. Despache o UUID novo.

Esperado no acompanhamento: `FALHA_DESPACHO`, ambulância e leito nulos. A ambulância 2 volta a `DISPONIVEL`. `L-UTI-02` continua `OCUPADO`. Não há `DESPACHO` para essa ocorrência. Repetir o despacho dela responde `200` e permanece `FALHA_DESPACHO`.

Sem ambulância

Ainda no aceite, com as duas ambulâncias livres de novo só se a anterior voltou. Reserve a enfermaria com a ambulância disponível. Abra uma emergência e despache.

Esperado: a enfermaria fica `DESPACHADA`. A emergência fica `FALHA_DESPACHO`, sem ambulância e sem leito. `L-EMG-01` continua `LIVRE`. A ocupação de ambulâncias fica `[]` enquanto as duas estiverem `DESPACHADA`.

Recusa e o mesmo leito

Carga limpa. Modo **Recusa**. Abra uma UTI e despache.

O acompanhamento oscila. Olhe a ocorrência e o MySQL por alguns segundos:

1. A ambulância fica `AGUARDANDO_LEITO`. Ela não volta para a base.
2. A ocorrência volta a `EM_DESPACHO`, com `leitoId` nulo.
3. `L-UTI-01` volta a `LIVRE` e `reserva_recusada` ganha `RECUSA_UNIDADE`.
4. Como não há outro livre de UTI, a saga reserva `L-UTI-01` de novo.

O ciclo continua enquanto o modo for **Recusa**. Confirme uma volta e clique **Aceite** ou **Aleatório** para parar. Encerrar o MS-Leitos com Ctrl+C também interrompe.

Outro leito na recusa

Carga limpa. **Criar L-UTI-03**. Modo **Recusa**. Abra uma UTI e despache.

A primeira reserva usa `L-UTI-01`. Na recusa, a seguinte prefere `L-UTI-03`. A notificação de despacho, quando a reserva seguinte existir, cita esse código. Pare o modo e apague `L-UTI-03` quando estiver `LIVRE`.

Silêncio e prazo

Carga limpa. Modo **Silêncio**. Abra uma UTI e despache.

Logo em seguida: ocorrência `DESPACHADA`, ambulância `DESPACHADA`, `L-UTI-01` `RESERVADO` sem confirmação. Há `DESPACHO`. Espere pouco mais de 2 minutos. O estado visível volta a `DESPACHADA` / `RESERVADO`, porque o único livre é pedido de novo. A prova é `confirmado` em `0` no MySQL, uma linha `EXPIRACAO` em `reserva_recusada` e o par `EXPIRACAO` depois `DESPACHO` nas notificações. Pare o modo para o ciclo não continuar.

Falta de leito

Carga limpa. Arme a trava. Modo **Recusa**. Abra uma UTI e despache.

A tentativa 1 reserva. A seguinte não acha vaga: a ambulância permanece fora de `DISPONIVEL`, a ocorrência fica `EM_DESPACHO` e as notificações ganham `FALTA_LEITO`.

Circuito

Gateway no ar. Ctrl+C só na porta 8095. Duas consultas de UTI em `503` e uma abertura em `201`. Suba o MS-Leitos. A consulta volta a `200`.

7. O que conferir fora da tela

A tela não mostra fila, chave do Redis nem a coluna `confirmado`.

Prova	Onde
Oito filas em Ready, Unacked e Total `0`	`http://localhost:15672`, `guest` / `guest`
`frota:disponiveis` e `leitos:disponiveis:UTI`	Redis na porta 6379
`confirmado`, `reserva_recusada`	MySQL `omnileito_leitos`, porta 3310
Documento gravado	`mongosh` no database `omnileito_notificacoes`

O painel do RabbitMQ e o Mongo estão detalhados na seção 13 do roteiro.

8. O que a tela não faz

- Não encerra processo Java nem container. O circuito pede Ctrl+C no terminal da porta 8095.
- Não muda `EM_TRANSPORTE`, `CONCLUIDA`, `CANCELADA`, `EM_ATENDIMENTO` nem `INDISPONIVEL`.
- Não envia SMS, não abre mapa e não junta o Swagger no gateway. O Swagger de cada serviço segue em `http://localhost:<porta>/swagger-ui.html`.
- Não substitui o roteiro em curl nem os testes `./mvnw test`.